URL Shortener: Stage 1 Final Review (revision 3)

Project	`com.malinda:url-shortener` (`E:\Projects\URL Shortener\url-shortener`)
Code reviewed	Branch `dev`, HEAD `665908a` "Add integration test for short URL generation without trailing slash in base URL" (2026-11-09 00:45 +0530), **plus the author's uncommitted working-tree changes** to `pom.xml`, `UrlServiceIMPL.java` and `UrlService.java`. Every result below comes from that working tree
Review date	2026-11-09 (tests finished 01:02:44 +0530, curl checks 01:03 to 01:04 +0530)
Reviewer	Claude Code (Claude Opus 5.5), at the author's request
Compared against	The 5 November review item list supplied by the author (no earlier review file exists in `docs/reviews/`)
Supersedes	Revision 2 (2026-11-09 00:27, reviewing `6fea8bb` plus staged changes) and revision 1 (2026-11-09 00:05, reviewing `cd61f9d`). Every step was rerun for this revision; nothing is carried over without being re-executed

What changed since revision 2

- **Base-URL fix.** `UrlServiceIMPL` now builds every short URL with a new private `buildShortUrl(code)` (lines 61-67), which adds a `/` when `app.base-url` lacks one. Revision 2's finding C1 is resolved.
- **New test.** The reviewer added `ShortUrlBaseUrlTest` (`app.base-url=http://short.test`), and the author committed it in `665908a`. It failed against `6fea8bb` with `expected "http://short.test/Mb7vVc" but was "http://short.testMb7vVc"`, and it passes now.
- **Cleanups.**
  - Removed the dead `code.isEmpty()` check (revision 2's D5).
  - Removed the `maven-compiler-plugin` block with `--enable-preview` from `pom.xml` (revision 2's D6). The working-tree `pom.xml` is byte-identical to the original from `b042eb1` (git blob `22046e8`).
  - Removed all three unused imports (revision 2's S1): `HttpStatus` and `ResponseEntity` in `UrlServiceIMPL`, and `CreateUrlResponse` in `UrlService`.
- **Report renumbering.** The non-ASCII `Location` finding is now C1. The design findings were renumbered and one new low-severity design note was added (D5). The style findings are now S1 to S5.
- **Commits.** The author committed `UrlShortenerErrorCasesTest` and the 404 fix in `1607d0a`, and `ShortUrlBaseUrlTest` with the revision 2 PDF in `665908a`.

1. Summary

All five Stage 1 exit criteria pass. All 16 tests pass. Every contract case behaves as specified under curl, including an unknown code (404 with no stack trace). `shortUrl` is now correct with and without a trailing slash in `app.base-url`, and no client error produces a 5xx. One minor correctness point remains, and the exit checklist doesn't cover it: non-ASCII URLs come back percent-encoded in `Location` (C1). Of the 5 November items, 10 are fixed, 1 is partly fixed and 7 are still open. All 7 open items are design or naming/style items (section 11). I did not change any code under `src/main`, `pom.xml` or config in this session.

2. Stage 1 exit checklist

Criterion	Result	Evidence
All tests pass	**PASS**	`Tests run: 16, Failures: 0, Errors: 0, Skipped: 0`, `BUILD SUCCESS` (section 6)
Every contract case verified by curl	**PASS**	201 for a new URL, 200 with the same code for a duplicate, 400 for all 9 invalid inputs (empty body plus the 8 listed), 302 with an unchanged `Location` for a known code, and 404 for an unknown code. `shortUrl` is correct for both base-URL forms (section 7)
No 5xx for any client error	**PASS**	No 5xx in any curl check, required or extra (section 7.4)
No stack traces for 4xx cases	**PASS**	All 11 4xx cases logged nothing, or one `WARN` line with no stack trace. The server logs from all four runs contain 0 stack-trace frames (section 7.4)
Parked items left alone	**PASS**	Collision handling, the check-then-act race, the linear scan and the static generator are unchanged and left for later stages (section 10)

3. Environment

Item	Value	Source
Java	21.0.7 (Oracle HotSpot, build 21.0.7+8-LTS-245)	`java -version`, `mvnw -v`
Java target	21 (`<java.version>21</java.version>`, applied by the Boot parent; javac ran with `release 21`)	`pom.xml`, build log
Spring Boot	4.1.2 (`spring-boot-starter-parent`)	`pom.xml` line 8, startup banner `:: Spring Boot :: (v4.1.2)`
Build tool	Maven 3.10.0 via Maven Wrapper 3.3.4 (`mvnw`, `distributionType=only-script`)	`.mvn/wrapper/maven-wrapper.properties`, `mvnw -v`
Dependencies	`spring-boot-starter-webmvc`, `spring-boot-starter-webmvc-test` (test)	`pom.xml`
Surefire	3.5.6	build log
OS	Windows 11 (10.0.26200), curl 8.14.1	

`pom.xml` changes since the 5 November review. The only change is the removal of the `maven-compiler-plugin` block (`<source>21</source>`, `<target>21</target>`, `<compilerArgs>--enable-preview</compilerArgs>`). It is uncommitted in the working tree. Commit `607f837` (2026-11-02) had added that block. With it gone, the working-tree `pom.xml` is byte-identical to the original from `b042eb1` (git blob `22046e8`), and the build still compiles with `release 21`. Dependencies and versions are unchanged.

Other config: `.gitignore` gained `docs/reviews/stage-1-final-review.md` (committed in `1607d0a`), so git does not track this Markdown report; the PDF is tracked. `application.yaml` is unchanged.

4. File tree of `src/`

```
src/
    main/
        java/
            com/
                malinda/
                    url_shortener/
                        UrlShortenerApplication.java
                        controller/
```

```
                        UrlController.java
                    dto/
                        ApiErrorResponse.java
                        CreateUrlRequest.java
                        CreateUrlResponse.java
                        CreateUrlResult.java
                    exceptions/
                        GlobalExceptionHandler.java
                        InvalidCodeException.java
                        InvalidUrlException.java
                    service/
                        UrlService.java
                        IMPL/
                            UrlServiceIMPL.java
                    util/
                        ShortCodeGenerator.java
                        UrlValidator.java
        resources/
            application.yaml
            static/
            templates/
    test/
        java/
            com/
                malinda/
                    url_shortener/
                        ShortUrlBaseUrlTest.java
                        UrlShortenerAcceptanceTest.java
                        UrlShortenerApplicationTests.java
                        UrlShortenerErrorCasesTest.java
```

`resources/static/` and `resources/templates/` are empty directories. The reviewer created `UrlShortenerErrorCasesTest.java` (revision 1) and `ShortUrlBaseUrlTest.java` (between revisions 2 and 3) at the author's request; the author committed both. Everything else is the author's.

5. Full contents of every file under `src/main`

These are the verbatim working-tree contents at the time of testing.

5.1 `src/main/java/com/malinda/url_shortener/UrlShortenerApplication.java`

```java
package com.malinda.url_shortener;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class UrlShortenerApplication {

    public static void main(String[] args) {
            SpringApplication.run(UrlShortenerApplication.class, args);
    }

}
```

5.2 `src/main/java/com/malinda/url_shortener/controller/UrlController.java`

```java
package com.malinda.url_shortener.controller;

import com.malinda.url_shortener.dto.CreateUrlRequest;
import com.malinda.url_shortener.dto.CreateUrlResponse;
import com.malinda.url_shortener.dto.CreateUrlResult;
import com.malinda.url_shortener.service.UrlService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
```

```
12
13  @RestController
14  public class UrlController {
15
16      private final UrlService urlService;
17
18      public UrlController(UrlService urlService) {
19          this.urlService = urlService;
20      }
21
22      @PostMapping("/api/urls")
23      public ResponseEntity<CreateUrlResponse> createShortUrl(@RequestBody CreateUrlRequest request) {
24          CreateUrlResult response = urlService.createShortUrl(request.url());
25
26          if (response.created()){
27              return ResponseEntity
28                      .status(HttpStatus.CREATED)
29                      .body(response.response());
30          } else {
31              return ResponseEntity
32                      .status(HttpStatus.OK)
33                      .body(response.response());
34          }
35      }
36
37      @GetMapping("/{code}")
38      public  ResponseEntity<Void> redirectToOriginalUrl(@PathVariable String code) {
39          String response = urlService.redirectToOriginalUrl(code);
40          return  ResponseEntity
41                  .status(HttpStatus.FOUND)
42                  .location(URI.create(response))
43                  .build();
44      }
45  }
```

5.3 `src/main/java/com/malinda/url_shortener/dto/ApiErrorResponse.java`

```
1  package com.malinda.url_shortener.dto;
2
3  public record ApiErrorResponse(String message) {
4  }
```

5.4 `src/main/java/com/malinda/url_shortener/dto/CreateUrlRequest.java`

```
1  package com.malinda.url_shortener.dto;
2
3  public record CreateUrlRequest(String url) {}
```

5.5 `src/main/java/com/malinda/url_shortener/dto/CreateUrlResponse.java`

```
1  package com.malinda.url_shortener.dto;
2
3  public record CreateUrlResponse(String code, String shortUrl) {}
```

5.6 `src/main/java/com/malinda/url_shortener/dto/CreateUrlResult.java`

```
1  package com.malinda.url_shortener.dto;
2
3  public record CreateUrlResult(CreateUrlResponse response, boolean created) {
4  }
```

5.7 `src/main/java/com/malinda/url_shortener/exceptions/GlobalExceptionHandler.java`

```
1  package com.malinda.url_shortener.exceptions;
2
3  import com.malinda.url_shortener.dto.ApiErrorResponse;
```

```java
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(InvalidUrlException.class)
    public ResponseEntity<ApiErrorResponse> handleInvalidUrlException(InvalidUrlException e) {
        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(new ApiErrorResponse(e.getMessage()));
    }

    @ExceptionHandler(InvalidCodeException.class)
    public ResponseEntity<ApiErrorResponse> handleInvalidCodeException(InvalidCodeException e) {
        return  ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(new ApiErrorResponse(e.getMessage()));
    }
}
```

5.8 `src/main/java/com/malinda/url_shortener/exceptions/InvalidCodeException.java`

```java
package com.malinda.url_shortener.exceptions;

public class InvalidCodeException extends RuntimeException{
    public InvalidCodeException(String message) {
        super(message);
    }
}
```

5.9 `src/main/java/com/malinda/url_shortener/exceptions/InvalidUrlException.java`

```java
package com.malinda.url_shortener.exceptions;

public class InvalidUrlException extends RuntimeException{
    public InvalidUrlException(String message) {
        super(message);
    }
}
```

5.10 `src/main/java/com/malinda/url_shortener/service/UrlService.java`

```java
package com.malinda.url_shortener.service;

import com.malinda.url_shortener.dto.CreateUrlResult;

public interface UrlService {
    CreateUrlResult createShortUrl(String longUrl);

    String redirectToOriginalUrl(String code);
}
```

5.11 `src/main/java/com/malinda/url_shortener/service/IMPL/UrlServiceIMPL.java`

```java
package com.malinda.url_shortener.service.IMPL;

import com.malinda.url_shortener.dto.CreateUrlResponse;
import com.malinda.url_shortener.dto.CreateUrlResult;
import com.malinda.url_shortener.exceptions.InvalidCodeException;
import com.malinda.url_shortener.service.UrlService;
import com.malinda.url_shortener.util.ShortCodeGenerator;
import com.malinda.url_shortener.util.UrlValidator;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

```

```java
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class UrlServiceIMPL implements UrlService {

    private final Map<String, String> urlMappings = new ConcurrentHashMap<>();

    @Value("${app.base-url}")
    private String baseUrl;

    @Override
    public CreateUrlResult createShortUrl(String longUrl) {
        UrlValidator.validateUrl(longUrl);


        for (Map.Entry<String, String> entry : urlMappings.entrySet()) {

            if (entry.getValue().equals(longUrl)) {
                String existingCode = entry.getKey();

                return new CreateUrlResult(
                        new CreateUrlResponse(
                                existingCode,
                                buildShortUrl(existingCode)),
                        false
                );
            }
        }

        String code = ShortCodeGenerator.generateShortCode();
        urlMappings.put(code, longUrl);
        return new CreateUrlResult(
                new CreateUrlResponse(
                        code,
                        buildShortUrl(code)),
                true
        );
    }

    @Override
    public String redirectToOriginalUrl(String code) {
        if (!urlMappings.containsKey(code)) {
            throw new InvalidCodeException("Unknown code");
        } else {
            return urlMappings.get(code);
        }
    }

    private String buildShortUrl(String code) {
        String normalizedBaseUrl = baseUrl.endsWith("/")
                ? baseUrl
                : baseUrl + "/";

        return normalizedBaseUrl + code;
    }
}
```

5.12 `src/main/java/com/malinda/url_shortener/util/ShortCodeGenerator.java`

```java
package com.malinda.url_shortener.util;

import java.security.SecureRandom;

public class ShortCodeGenerator {
    private static final String CHARACTERS =
            "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

    private static final SecureRandom secureRandom = new SecureRandom();

    public static String generateShortCode() {
        StringBuilder shortCode = new StringBuilder();

        for (int i = 0; i < 6; i++) {
```

```
            int randomIndex = secureRandom.nextInt(CHARACTERS.length());
            shortCode.append(CHARACTERS.charAt(randomIndex));
        }

        return shortCode.toString();
    }
}
```

5.13 `src/main/java/com/malinda/url_shortener/util/UrlValidator.java`

```
package com.malinda.url_shortener.util;

import com.malinda.url_shortener.exceptions.InvalidUrlException;

import java.net.URI;

public class UrlValidator {
    public static void validateUrl(String longUrl) {
        if (longUrl == null || longUrl.isBlank()) {
            throw new InvalidUrlException("URL cannot be empty");
        }

        try {
            URI uri = URI.create(longUrl);

            String schema = uri.getScheme();

            if (schema == null || (!schema.equalsIgnoreCase("http") && !schema.equalsIgnoreCase("https")) ||
uri.getHost() == null) {
                throw new InvalidUrlException("Invalid URL format");
            }
        } catch (IllegalArgumentException e) {
            throw new InvalidUrlException("Malformed URL");
        }
    }
}
```

5.14 `src/main/resources/application.yaml`

```
spring:
  application:
    name: url-shortener

server:
  port: 8081

app:
  base-url: http://localhost:8081/
```

Line numbers are added for reference and are not part of the files. The findings in section 8 cite these numbers.

6. Test files, commands and results

6.1 Test files

`UrlShortenerAcceptanceTest.java` (existing, author's, unchanged since `234e362` 2026-11-02):

```
package com.malinda.url_shortener;

import com.jayway.jsonpath.JsonPath;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;
```

```
import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * Black-box acceptance test for the Stage 1 API contract:
 *   POST /api/urls {"url": "..."} -> 201 {"code": "...", "shortUrl": "<base>/<code>"}
 *   GET  /{code}                  -> 3xx, Location: original url | 404 if unknown
 * Exercises the app only through HTTP (MockMvc does not follow redirects).
 */
@SpringBootTest
@AutoConfigureMockMvc
class UrlShortenerAcceptanceTest {

    private static final String TRACKING_URL =
            "https://shop.example.com/products/12345?utm_source=sms&utm_campaign=launch";
    private static final String ENCODED_URL = "https://example.com/search?q=hello%20world";

    @Autowired
    private MockMvc mockMvc;

    /** POSTs a URL, asserts the creation contract, and returns the code. */
    private String shorten(String longUrl) throws Exception {
        String body = "{\"url\": " + jsonString(longUrl) + "}";
        MvcResult result = mockMvc.perform(post("/api/urls")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(body))
                .andExpect(status().isCreated())
                .andReturn();

        String json = result.getResponse().getContentAsString();
        String code = JsonPath.read(json, "$.code");
        String shortUrl = JsonPath.read(json, "$.shortUrl");

        assertThat(code).isNotBlank();
        assertThat(shortUrl).endsWith("/" + code);
        return code;
    }

    private static String jsonString(String s) {
        return "\"" + s.replace("\\", "\\\\").replace("\"", "\\\"") + "\"";
    }

    @Test
    void createReturns201WithCodeAndShortUrl() throws Exception {
        shorten("https://example.com/some/page");
    }

    @Test
    void shortCodeRedirectsToExactOriginalUrl() throws Exception {
        String code = shorten(TRACKING_URL);

        mockMvc.perform(get("/" + code))
                .andExpect(status().is3xxRedirection())
                .andExpect(header().string("Location", TRACKING_URL));
    }

    @Test
    void percentEncodedUrlIsNotDoubleEncoded() throws Exception {
        String code = shorten(ENCODED_URL);

        mockMvc.perform(get("/" + code))
                .andExpect(status().is3xxRedirection())
                .andExpect(header().string("Location", ENCODED_URL));
    }

    @Test
    void differentUrlsGetDifferentCodesAndRedirectToTheirOwnOriginal() throws Exception {
        String urlA = "https://a.example.com/first";
        String urlB = "https://b.example.com/second";

        String codeA = shorten(urlA);
        String codeB = shorten(urlB);
```

```
        assertThat(codeA).isNotEqualTo(codeB);

        mockMvc.perform(get("/" + codeA))
                .andExpect(status().is3xxRedirection())
                .andExpect(header().string("Location", urlA));
        mockMvc.perform(get("/" + codeB))
                .andExpect(status().is3xxRedirection())
                .andExpect(header().string("Location", urlB));
    }

    @Test
    void unknownCodeReturns404() throws Exception {
        mockMvc.perform(get("/neverCreatedCode123"))
                .andExpect(status().isNotFound());
    }
}
```

`UrlShortenerApplicationTests.java` (existing, generated by Spring Initializr):

```
package com.malinda.url_shortener;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class UrlShortenerApplicationTests {

        @Test
        void contextLoads() {
        }

}
```

`UrlShortenerErrorCasesTest.java` (**created by the reviewer in revision 1**, committed by the author in `1607d0a`; unchanged since). It tests over HTTP only and uses hosts under `errcases.example.com`:

```
package com.malinda.url_shortener;

import com.jayway.jsonpath.JsonPath;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * Black-box error-case and duplicate tests for the Stage 1 API contract:
 *   POST /api/urls new URL           -> 201 {"code", "shortUrl"}
 *   POST /api/urls already shortened -> 200 {"code", "shortUrl"} with the existing code
 *   POST /api/urls missing / blank / malformed / not http(s) -> 400
 * HTTP only (MockMvc). All test classes share one in-memory store, so every URL here
 * uses a host under errcases.example.com that appears in no other test.
 */
@SpringBootTest
@AutoConfigureMockMvc
class UrlShortenerErrorCasesTest {

    @Autowired
    private MockMvc mockMvc;

    private MvcResult postBody(String body) throws Exception {
        return mockMvc.perform(post("/api/urls")
                        .contentType(MediaType.APPLICATION_JSON)
```

```
                        .content(body))
                .andReturn();
    }

    @Test
    void sameUrlTwiceReturns201ThenExisting200WithSameCodeAndShortUrl() throws Exception {
        String body = "{\"url\": \"https://dup.errcases.example.com/page?x=1\"}";

        MvcResult first = mockMvc.perform(post("/api/urls")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(body))
                .andExpect(status().isCreated())
                .andReturn();
        MvcResult second = mockMvc.perform(post("/api/urls")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(body))
                .andExpect(status().isOk())
                .andReturn();

        String firstJson = first.getResponse().getContentAsString();
        String secondJson = second.getResponse().getContentAsString();
        String firstCode = JsonPath.read(firstJson, "$.code");
        String firstShortUrl = JsonPath.read(firstJson, "$.shortUrl");

        assertThat(firstCode).isNotBlank();
        assertThat((String) JsonPath.read(secondJson, "$.code")).isEqualTo(firstCode);
        assertThat((String) JsonPath.read(secondJson, "$.shortUrl")).isEqualTo(firstShortUrl);
    }

    @ParameterizedTest(name = "[{index}] {0} -> 400")
    @ValueSource(strings = {
            "{}",
            "{\"url\": \"\"}",
            "{\"url\": \"   \"}",
            "{\"url\": \"not-a-url\"}",
            "{\"url\": \"https://\"}",
            "{\"url\": \"javascript:alert(1)\"}",
            "{\"url\": \"ftp://errcases.example.com/file\"}",
            "{\"url\": \"https://errcases.example.com/a b\"}"
    })
    void invalidInputReturns400(String body) throws Exception {
        assertThat(postBody(body).getResponse().getStatus()).isEqualTo(400);
    }
}
```

`ShortUrlBaseUrlTest.java` (**created by the reviewer at the author's request** after revision 2, committed by the author in `665908a`). It runs in its own application context with `app.base-url=http://short.test`, and its URL `https://baseurl-test.example.com/only-here` appears in no other file:

```
package com.malinda.url_shortener;

import com.jayway.jsonpath.JsonPath;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * shortUrl must be base-url + "/" + code even when app.base-url has no trailing slash.
 * HTTP only (MockMvc). Uses a URL that appears in no other test.
 */
@SpringBootTest(properties = "app.base-url=http://short.test")
@AutoConfigureMockMvc
class ShortUrlBaseUrlTest {

    @Autowired
```

```
    private MockMvc mockMvc;

    @Test
    void shortUrlIsBaseUrlSlashCodeWhenBaseUrlHasNoTrailingSlash() throws Exception {
        MvcResult result = mockMvc.perform(post("/api/urls")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"url\": \"https://baseurl-test.example.com/only-here\"}"))
                .andExpect(status().isCreated())
                .andReturn();

        String json = result.getResponse().getContentAsString();
        String code = JsonPath.read(json, "$.code");
        String shortUrl = JsonPath.read(json, "$.shortUrl");

        assertThat(code).isNotBlank();
        assertThat(shortUrl).isEqualTo("http://short.test/" + code);
    }
}
```

6.2 Command

```
cd url-shortener
./mvnw -B clean test
```

Result: `Tests run: 16, Failures: 0, Errors: 0, Skipped: 0`, `BUILD SUCCESS` (finished 2026-11-09T01:02:44+05:30, 55.0 s). `clean` removed the previous build and test reports first, so all classes were recompiled: 13 main and 4 test source files.

6.3 Per-test results (from `target/surefire-reports/TEST-*.xml`)

#	Class	Test	Result
1	ShortUrlBaseUrlTest	shortUrlIsBaseUrlSlashCodeWhenBaseUrlHasNoTrailingSlash	PASS (failed against `6fea8bb`)
2	UrlShortenerAcceptanceTest	createReturns201WithCodeAndShortUrl	PASS
3	UrlShortenerAcceptanceTest	shortCodeRedirectsToExactOriginalUrl	PASS
4	UrlShortenerAcceptanceTest	percentEncodedUrlIsNotDoubleEncoded	PASS
5	UrlShortenerAcceptanceTest	differentUrlsGetDifferentCodesAndRedirectToTheirOwnOriginal	PASS
6	UrlShortenerAcceptanceTest	unknownCodeReturns404	PASS (errored in revision 1)
7	UrlShortenerApplicationTests	contextLoads	PASS
8	UrlShortenerErrorCasesTest	sameUrlTwiceReturns201ThenExisting200WithSameCodeAndShortUrl	PASS
9	UrlShortenerErrorCasesTest	invalidInputReturns400 [1] `{}`	PASS
10	UrlShortenerErrorCasesTest	invalidInputReturns400 [2] `{"url": ""}`	PASS
11	UrlShortenerErrorCasesTest	invalidInputReturns400 [3] `{"url": "   "}`	PASS
12	UrlShortenerErrorCasesTest	invalidInputReturns400 [4] `{"url": "not-a-url"}`	PASS
13	UrlShortenerErrorCasesTest	invalidInputReturns400 [5] `{"url": "https://"}`	PASS

#	Class	Test	Result
14	UrlShortenerErrorCasesTest	invalidInputReturns400 [6] `{"url": "javascript:alert(1)"}`	PASS
15	UrlShortenerErrorCasesTest	invalidInputReturns400 [7] `{"url": "ftp://errcases.example.com/file"}`	PASS
16	UrlShortenerErrorCasesTest	invalidInputReturns400 [8] `{"url": "https://errcases.example.com/a b"}`	PASS

Each surefire XML file contains 0 `<error>` and 0 `<failure>` elements.

6.4 Notes on the tests (no existing test edited)

- No test contradicts the contract.
- Several acceptance tests use `status().is3xxRedirection()`, which is looser than the contract's 302: a 301 or 307 would also pass. The class Javadoc also says "3xx". This is a gap, not a wrong assertion.
- The acceptance test's `shorten()` checks only that `shortUrl` ends with `"/" + code`. The base-URL prefix is now pinned exactly by `ShortUrlBaseUrlTest` for the no-slash form. The trailing-slash form is used by every other test class through the yaml value, but no test asserts its full `shortUrl` string; curl check 6b covers it.
- `unknownCodeReturns404` checks only the status code, not the `{"message":"Unknown code"}` body. That's fine for the contract, which specifies only the status.
- The build log shows the JVM warnings "A Java agent has been loaded dynamically (byte-buddy-agent)" and "Sharing is only supported for boot loader classes". These are Mockito/JDK 21 notices and do not affect the results.

7. curl transcripts

Method: I rebuilt the jar from the current working tree with `./mvnw -B -q -DskipTests package` and ran it in the background with `java -jar target/url-shortener-0.0.1-SNAPSHOT.jar --server.port=<port> --app.base-url=<base>`. I ran the curl checks with `curl -s -i`, then stopped the process. Afterwards `netstat` showed no listeners on ports 18080 to 18089. The `Date`, `Connection`, `Keep-Alive`, `Transfer-Encoding` and `Vary` headers are trimmed. After each request, the `[server log: ...]` line reports what the app log gained during that request.

7.1 Checks 1 to 5 (port 18081, `--app.base-url=http://localhost:18081/`)

```
== App started: port 18081, --app.base-url=http://localhost:18081/ (pid 4895)

### 1a. Create a new link
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d
'{"url":"https://curl.example.com/articles/42?ref=manual"}'
HTTP/1.1 201
Content-Type: application/json

{"code":"CeDwlh","shortUrl":"http://localhost:18081/CeDwlh"}

[server log: no new lines; no stack trace]

### 1b. Follow it (code CeDwlh)
$ curl -i http://localhost:18081/CeDwlh
HTTP/1.1 302
Location: https://curl.example.com/articles/42?ref=manual
Content-Length: 0


[server log: no new lines; no stack trace]

### 2. Submit the same URL again (expect 200, code CeDwlh)
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d
'{"url":"https://curl.example.com/articles/42?ref=manual"}'
HTTP/1.1 200
```

```
Content-Type: application/json

{"code":"CeDwlh","shortUrl":"http://localhost:18081/CeDwlh"}

[server log: no new lines; no stack trace]

### 3a. Create the percent-encoded URL
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d
'{"url":"https://example.com/search?q=hello%20world"}'
HTTP/1.1 201
Content-Type: application/json

{"code":"v8czLf","shortUrl":"http://localhost:18081/v8czLf"}

[server log: no new lines; no stack trace]

### 3b. Follow it (code v8czLf)
$ curl -i http://localhost:18081/v8czLf
HTTP/1.1 302
Location: https://example.com/search?q=hello%20world
Content-Length: 0


[server log: no new lines; no stack trace]

### 4. Unknown code
$ curl -i http://localhost:18081/zzzzzz
HTTP/1.1 404
Content-Type: application/json

{"message":"Unknown code"}

[server log: no new lines; no stack trace]

### 5a. Empty body (no content)
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json'
HTTP/1.1 400
Content-Type: application/json

{"timestamp":"2026-11-08T19:33:17.717Z","status":400,"error":"Bad Request","path":"/api/urls"}

[server log: no stack trace; single-line entry:]
2026-11-09T01:03:17.716+05:30  WARN 14848 --- [url-shortener] [io-18081-exec-5]
.w.s.m.s.DefaultHandlerExceptionResolver : Resolved
[org.springframework.http.converter.HttpMessageNotReadableException: Required request body is missing: public
org.springframework.http.ResponseEntity<com.malinda.url_shortener.dto.CreateUrlResponse>
com.malinda.url_shortener.controller.UrlController.createShortUrl(com.malinda.url_shortener.dto.CreateUrlRequest)]

### 5b. {}
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{}'
HTTP/1.1 400
Content-Type: application/json

{"message":"URL cannot be empty"}

[server log: no new lines; no stack trace]

### 5c. empty string
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url": ""}'
HTTP/1.1 400
Content-Type: application/json

{"message":"URL cannot be empty"}

[server log: no new lines; no stack trace]

### 5d. blank
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url": "   "}'
HTTP/1.1 400
Content-Type: application/json

{"message":"URL cannot be empty"}

[server log: no new lines; no stack trace]

### 5e. not-a-url
```

```
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url": "not-a-url"}'
HTTP/1.1 400
Content-Type: application/json

{"message":"Invalid URL format"}

[server log: no new lines; no stack trace]

### 5f. https:// (no host)
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url": "https://"}'
HTTP/1.1 400
Content-Type: application/json

{"message":"Malformed URL"}

[server log: no new lines; no stack trace]

### 5g. javascript scheme
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url":
"javascript:alert(1)"}'
HTTP/1.1 400
Content-Type: application/json

{"message":"Invalid URL format"}

[server log: no new lines; no stack trace]

### 5h. ftp scheme
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url":
"ftp://errcases.example.com/file"}'
HTTP/1.1 400
Content-Type: application/json

{"message":"Invalid URL format"}

[server log: no new lines; no stack trace]

### 5i. space in path
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url":
"https://errcases.example.com/a b"}'
HTTP/1.1 400
Content-Type: application/json

{"message":"Malformed URL"}

[server log: no new lines; no stack trace]

== Additional probes (outside the required list)

### X1. malformed JSON
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d '{"url":'
HTTP/1.1 400
Content-Type: application/json

{"timestamp":"2026-11-08T19:33:25.660Z","status":400,"error":"Bad Request","path":"/api/urls"}

[server log: no stack trace; single-line entry:]
2026-11-09T01:03:25.660+05:30  WARN 14848 --- [url-shortener] [io-18081-exec-4]
.w.s.m.s.DefaultHandlerExceptionResolver : Resolved
[org.springframework.http.converter.HttpMessageNotReadableException: JSON parse error: Unexpected end-of-input
within/between Object entries]

### X2. non-ASCII path, sent as a UTF-8 body file containing: {"url":"https://probe.example.com/café"}
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' --data-binary @body-cafe.json
HTTP/1.1 201
Content-Type: application/json

{"code":"Gy4wGJ","shortUrl":"http://localhost:18081/Gy4wGJ"}

[server log: no new lines; no stack trace]

### X2b. Follow it (code Gy4wGJ); stored URL is https://probe.example.com/café
$ curl -i http://localhost:18081/Gy4wGJ
HTTP/1.1 302
Location: https://probe.example.com/caf%C3%A9
Content-Length: 0
```

```
[server log: no new lines; no stack trace]

### X3. trailing-slash variant of an existing URL
$ curl -i -X POST http://localhost:18081/api/urls -H 'Content-Type: application/json' -d
'{"url":"https://curl.example.com/articles/42/?ref=manual"}'
HTTP/1.1 201
Content-Type: application/json

{"code":"stf7cf","shortUrl":"http://localhost:18081/stf7cf"}

[server log: no new lines; no stack trace]

== App stopped
```

Number of stack-trace frames (`at ...` lines) in the whole server log for this run: 0.

Note on X2: I sent the `é` URL as a UTF-8 body file. On this Windows machine, passing `é` as a command-line argument would make curl.exe send a Latin-1 byte, which isn't valid UTF-8 JSON.

7.2 Check 6a: `--app.base-url=http://localhost:18082` (no trailing slash)

```
== App started: port 18082, --app.base-url=http://localhost:18082 (pid 5130)

### 6. Create with this base-url
$ curl -i -X POST http://localhost:18082/api/urls -H 'Content-Type: application/json' -d
'{"url":"https://baseurl.example.com/page"}'
HTTP/1.1 201
Content-Type: application/json

{"code":"UYa0sI","shortUrl":"http://localhost:18082/UYa0sI"}

[server log: no new lines; no stack trace]

### 6b. Follow it (code UYa0sI)
$ curl -i http://localhost:18082/UYa0sI
HTTP/1.1 302
Location: https://baseurl.example.com/page
Content-Length: 0


[server log: no new lines; no stack trace]

== App stopped
```

Result: correct. `shortUrl` is `http://localhost:18082/UYa0sI`. In revisions 1 and 2 this form returned `http://localhost:18082<code>`, with no `/` between the port and the code.

7.3 Check 6b: `--app.base-url=http://localhost:18083/` (trailing slash)

```
== App started: port 18083, --app.base-url=http://localhost:18083/ (pid 5166)

### 6. Create with this base-url
$ curl -i -X POST http://localhost:18083/api/urls -H 'Content-Type: application/json' -d
'{"url":"https://baseurl.example.com/page"}'
HTTP/1.1 201
Content-Type: application/json

{"code":"6Hq8qd","shortUrl":"http://localhost:18083/6Hq8qd"}

[server log: no new lines; no stack trace]

### 6b. Follow it (code 6Hq8qd)
$ curl -i http://localhost:18083/6Hq8qd
HTTP/1.1 302
Location: https://baseurl.example.com/page
Content-Length: 0
```

```
[server log: no new lines; no stack trace]

== App stopped
```

Result: correct. `shortUrl` is `http://localhost:18083/6Hq8qd`, with no doubled slash.

7.4 Stack traces for 4xx cases

Case	Status	Stack trace in server log?
4 unknown code	404 `{"message":"Unknown code"}`	No, nothing logged
5a empty body	400	No. One `WARN` line from `DefaultHandlerExceptionResolver` ("Required request body is missing")
5b `{}`	400	No, nothing logged
5c `""`	400	No, nothing logged
5d `"  "`	400	No, nothing logged
5e `not-a-url`	400	No, nothing logged
5f `https://`	400	No, nothing logged
5g `javascript:alert(1)`	400	No, nothing logged
5h `ftp://...`	400	No, nothing logged
5i `.../a b`	400	No, nothing logged
X1 malformed JSON (extra)	400	No. One `WARN` line ("JSON parse error")
7.5 `GET /` (extra)	404	No (0 frames)

7.5 Extra: `GET /` (no code), default config on port 18084

```
$ curl -i http://localhost:18084/
HTTP/1.1 404
Content-Type: application/json

{"timestamp":"2026-11-08T19:33:54.898Z","status":404,"error":"Not Found","path":"/"}

[server log stack frames: 0]
```

8. Review findings by severity

Order: correctness, then concurrency/data integrity, then performance, then design, then style. Each finding is tagged **[observed]** (reproduced by a test or curl) or **[code reading]**. Line numbers refer to section 5.

8.0 Verification of the fixes since revision 1

- **Unknown code gives 404.** `UrlServiceIMPL.redirectToOriginalUrl` (lines 53-59) throws `InvalidCodeException("Unknown code")` at line 55. `GlobalExceptionHandler.handleInvalidCodeException` (lines 19-24) maps it to `404 NOT_FOUND`.
    - Evidence: test #6 passes, and curl check 4 returns 404 with nothing logged. [observed]
    - The controller now only receives stored URLs, and those already passed `URI.create` during validation, so the `URI.create` at controller line 42 cannot throw for them. [code reading]

- **Base URL with or without a trailing slash.** `buildShortUrl` (lines 61-67) appends `/` only when `baseUrl` doesn't already end with one. Both the new-code path (line 47) and the duplicate path (line 36) use it.
    - Evidence: test #1 passes, curl 6a gives `http://localhost:18082/UYa0sI`, and curl 6b gives `http://localhost:18083/6Hq8qd`. [observed]
- **Cleanups.** The dead `code.isEmpty()` check is gone (line 54), and so is the `--enable-preview` block.
    - Evidence: the build log shows javac with `[debug parameters release 21]`, and all tests pass. [observed]
    - A scan of every `src/main` file found no unused imports. [code reading]

8.1 Correctness

C1. A non-ASCII URL comes back changed in `Location`. [observed; mechanism by code reading]

- *Where:* `UrlController.redirectToOriginalUrl`, line 42 (`.location(URI.create(response))`).
- *What:* `https://probe.example.com/café` was accepted (201), but the redirect returned `Location: https://probe.example.com/caf%C3%A9` (curl X2/X2b). Spring writes the `Location` header from the `URI` in its ASCII form, which percent-encodes non-ASCII characters.
- *Why it matters:* The contract says "Location: the original URL, unchanged". Browsers treat the two forms as equivalent, and HTTP headers are ASCII-only anyway, so the risk is low. But the API currently accepts input that it cannot return unchanged.
- *Fix direction:* Decide the contract for non-ASCII input: reject it at validation, or document that it is stored or returned in ASCII-encoded form. Then add a test that pins whichever behaviour you choose.

8.2 Concurrency / data integrity

No findings outside the parked items. `ConcurrentHashMap` makes individual map operations thread-safe. The compound read-then-write in `createShortUrl` and possible code collisions are parked (section 10).

8.3 Performance

No findings outside the parked items. The duplicate lookup's linear scan is parked (section 10).

8.4 Design

D1. The service builds the API response DTO and the short-URL string. [code reading]

- *Where:* `UrlServiceIMPL.createShortUrl` (lines 33-49), `buildShortUrl` (lines 61-67), and `CreateUrlResult` (wraps `dto.CreateUrlResponse`).
- *What:* The service returns `CreateUrlResult(CreateUrlResponse, boolean)` and formats the public URL itself, so the HTTP response shape and the public-host concern live in the service layer.
- *Why it matters:* A change to the response JSON or to the public host forces a service change, and the service cannot be reused or tested without the web-facing DTO. It is also why `baseUrl` has to be injected into the service at all.
- *Fix direction:* Have the service return a domain result (the code and whether it was newly created). Let the web layer turn it into `CreateUrlResponse` and build `shortUrl`.

D2. `baseUrl` uses field injection. [code reading]

- *Where:* `UrlServiceIMPL`, lines 20-21 (`@Value("${app.base-url}") private String baseUrl;`).
- *What and why:* The field cannot be `final`, its dependency is hidden from the constructor, and a plain unit test cannot set it without reflection or a Spring context. `ShortUrlBaseUrlTest` needs a whole application context just to vary it. The controller already uses constructor injection correctly, so the two classes are inconsistent.

- *Fix direction:* Inject the value through the constructor (or a typed configuration-properties object), in line with D1.

D3. Two different error-response shapes. [observed]

- *Where:* `GlobalExceptionHandler` (handles only `InvalidUrlException` and `InvalidCodeException`).
- *What:* Validation errors and unknown codes return `{"message":"..."}`. An empty body or malformed JSON returns Spring's default `{"timestamp","status","error","path"}` (checks 5a and X1), as does an unmatched path (7.5).
- *Why it matters:* Clients must parse two different 400 bodies from the same endpoint.
- *Fix direction:* Choose one error format and make the advice produce it for request-body read failures too (Spring's `ProblemDetail` support is one option).

D4. The service method name `redirectToOriginalUrl` is misleading. [code reading]

- *Where:* `UrlService` line 8 and `UrlServiceIMPL` line 53. The controller method uses the same name at line 38.
- *What and why:* The service does not redirect; it looks up a stored URL and throws when the code is missing. The name hides that contract from callers.
- *Fix direction:* Name the service method for what it does (a lookup or resolve). The controller handler can keep a redirect-oriented name.

D5. Base-URL normalisation runs on every request instead of once. [code reading] (low; new in this revision)

- *Where:* `UrlServiceIMPL.buildShortUrl`, lines 61-67.
- *What:* The configured value never changes at runtime, but it is checked and possibly re-concatenated on every create call. The normalisation lives in the same method as code formatting.
- *Why it matters:* The cost is small. The issue is that the rule "the base URL is stored in canonical form" is not established anywhere, so another future use of `baseUrl` would need to repeat it.
- *Fix direction:* Normalise once where the value enters the class (fits naturally with D2's constructor injection), and keep the per-code step a plain join.

8.5 Style

- **S1. Wildcard import.** [code reading] `UrlController` line 9 (`org.springframework.web.bind.annotation.*`). It hides which annotations the class uses. Use explicit imports.
- **S2. Double spaces and spacing.** [code reading]
  - `UrlController` line 38 (`public  ResponseEntity`), line 40 (`return  ResponseEntity`) and line 26 (`created()){`).
  - `GlobalExceptionHandler` line 21 (`return  ResponseEntity`).
  - `InvalidUrlException` and `InvalidCodeException` both have `RuntimeException{` at line 3.
  - `UrlServiceIMPL` lines 26-27 are two blank lines.
  - Running the IDE formatter would fix all of these.
- **S3. All-caps `IMPL` naming.** [code reading] Package `service.IMPL` and class `UrlServiceIMPL`. Java convention is lowercase package names and CamelCase class names (for example `service.impl` and `UrlServiceImpl`, or a name that describes the implementation, such as an in-memory one).
- **S4. Magic number 6.** [code reading] `ShortCodeGenerator.generateShortCode`, line 14. The code length is a domain rule and should be a named constant.
- **S5. Naming nits.** [code reading]
  - `InvalidCodeException` is thrown for a code that is well-formed but not stored, and it maps to 404, so a name meaning "not found" would match its meaning and status better.
  - `UrlValidator` line 16 names the variable `schema`; the URI concept is "scheme".

    - `UrlController` names both the `CreateUrlResult` (line 24) and the looked-up URL string (line 39) `response`, although neither is a response.
    - The 201/200 branch at lines 26-34 repeats the `.body(response.response())` chain where only the status differs.

9. What's done well

- **Both Stage 1 bugs are fixed narrowly and in the right place.**
    - The unknown-code fix is a dedicated exception plus a handler method in the existing `@RestControllerAdvice`, so the controller has no special cases and nothing is logged.
    - The base-URL fix is a single helper that both the new-code path and the duplicate path go through, so the two responses can't drift apart.
- **Cleanup alongside the fixes.** The dead check, the unnecessary `--enable-preview` block and all the unused imports are gone, leaving no unused imports in `src/main` and a `pom.xml` back to its original form.
- **Validation is in one place** (`UrlValidator`). It has a dedicated `InvalidUrlException`, mapped to 400. All 8 invalid inputs and the empty body return 400 with a clear message and **no stack trace**.
- **Duplicate handling matches the contract.** A resubmitted URL returns 200 with the same `code` and `shortUrl` (test #8 and curl check 2), and the `created` flag keeps the 201/200 decision explicit.
- **Redirects are correct for normal URLs.** They return 302, and query strings and percent-encoding are preserved exactly (`?q=hello%20world`, `?utm_source=...`). There is no double-encoding.
- The scheme check is case-insensitive and correctly rejects `javascript:` and `ftp:`.
- **Constructor injection** in `UrlController`, and a service interface the controller depends on.
- **Records** for every DTO, so they are immutable and concise.
- `SecureRandom` for codes and `ConcurrentHashMap` for the store are sensible choices for this stage.
- The endpoint (`/api/urls`), the request field (`url`) and the response fields (`code`, `shortUrl`) exactly match the contract.

10. Future-stage flags

- A newly generated code can equal an existing code, and the new mapping silently replaces the old one.
- Two simultaneous requests for the same new URL can both get 201 with different codes.
- Duplicate lookup time grows linearly with the number of stored links.
- Code generation is a static call, so tests cannot control which code is produced.

11. Comparison with the 5 November review

No 5 November review file exists in `docs/reviews/`, so this section uses the list supplied by the author. The evidence covers the commits since then, `865430c` (2026-11-06), `cd61f9d` (2026-11-08), `6fea8bb`, `1607d0a` and `665908a` (2026-11-09), plus the uncommitted working-tree changes.

#	5 November item	Status	Evidence
1	Unknown code returned 500	**Fixed**	Now 404 `{"message":"Unknown code"}` with no stack trace (curl check 4; test #6). `UrlServiceIMPL` line 55, `GlobalExceptionHandler` lines 19-24
2	Blank or missing url returned 500	**Fixed**	`{}`, `""` and `"  "` each give 400 `URL cannot be empty` (`UrlValidator` lines 9-11; tests #9-11; curl 5b-5d). An empty body also gives 400 (5a)

#	5 November item	Status	Evidence
3	Duplicate URL returned 500	**Fixed**	`865430c` still threw a bare `RuntimeException` for duplicates. The current code returns 200 with the existing code (`UrlServiceIMPL` lines 28-40; test #8; curl check 2)
4	URL format not validated	**Fixed**	`UrlValidator` checks the URI syntax, an http(s) scheme and a non-null host. All 8 invalid inputs give 400 (tests #9-16; curl 5b-5i). Edge case: non-ASCII input is accepted but returned encoded (C1)
5	An empty-code check that can never be true	**Fixed**	`code.isEmpty() ||` is removed; line 54 is now `if (!urlMappings.containsKey(code))` (working tree)
6	The lookup returned an error string as data	**Fixed**	`return "Invalid code";` is replaced by `throw new InvalidCodeException("Unknown code");` (`UrlServiceIMPL` line 55)
7	Bare RuntimeExceptions with nothing mapping them to statuses	**Fixed**	The bare `throw new RuntimeException()` calls from `865430c` are gone. Both remaining application exceptions are specific types mapped in `GlobalExceptionHandler`: `InvalidUrlException` to 400 and `InvalidCodeException` to 404. No client error produced a 5xx (section 7.4)
8	shortUrl depended on a trailing slash in base-url	**Fixed**	`buildShortUrl` (`UrlServiceIMPL` lines 61-67) normalises the base URL. `ShortUrlBaseUrlTest` passes, and curl gives `http://localhost:18082/UYa0sI` (no slash) and `http://localhost:18083/6Hq8qd` (slash) (7.2, 7.3)
9	The service returned the API response DTO and built the short-URL string	**Open**	The service still builds `CreateUrlResponse(code, buildShortUrl(code))`, wrapped in `CreateUrlResult` (D1)
10	Field injection for baseUrl	**Open**	`UrlServiceIMPL` lines 20-21 are unchanged (D2)
11	Misleading method name redirectToOriginalUrl	**Open**	`UrlService` line 8 and `UrlServiceIMPL` line 53 are unchanged (D4)
12	--enable-preview in the pom	**Fixed**	The `maven-compiler-plugin` block is removed (working tree). `pom.xml` is byte-identical to `b042eb1`, and javac runs with `release 21` (section 3)
13	Unused imports	**Fixed**	`HttpStatus` and `ResponseEntity` are removed from `UrlServiceIMPL`, and `CreateUrlResponse` from `UrlService` (working tree). A scan of every `src/main` file finds no unused imports
14a	All-caps IMPL naming	**Open**	`service/IMPL/UrlServiceIMPL.java` (S3)
14b	Double spaces	**Open**	`UrlController` lines 38 and 40, and `GlobalExceptionHandler` line 21 (S2)
14c	Wildcard import	**Open**	`UrlController` line 9 (S1)
14d	Magic number 6	**Open**	`ShortCodeGenerator` line 14 (S4)

#	5 November item	Status	Evidence
15	No tests for duplicates or invalid input	**Partly**	These cases (and the base-URL case) are now covered and committed. However, the reviewer wrote the covering tests (`UrlShortenerErrorCasesTest`, `ShortUrlBaseUrlTest`) at the author's request; the author committed them in `1607d0a` and `665908a`. The author's own test file, `UrlShortenerAcceptanceTest`, is unchanged since `234e362` (2026-11-02)

Tally: **10 fixed, 1 partly, 7 open** (18 rows, counting 14a-14d separately). All 7 open items are design (9, 10, 11) or naming/style (14a-14d) issues; none affects the contract.

12. Facts only

Redirect status. `302 Found` (`HttpStatus.FOUND`, `UrlController` line 41). Observed in curl checks 1b, 3b and 6b.

How short codes are generated. `ShortCodeGenerator.generateShortCode()` is a static method. It loops 6 times, and on each pass picks `CHARACTERS.charAt(secureRandom.nextInt(62))` from `0-9a-zA-Z` (62 characters) using one shared static `SecureRandom`. The result is 6 characters, giving 62^6 (about 5.68 x 10^10) possible codes. No uniqueness check is made before the code is stored.

How `shortUrl` is built. `buildShortUrl(code)` (`UrlServiceIMPL` lines 61-67) takes the configured `app.base-url`. If it does not end with `/`, one `/` is appended; the code is then appended. Examples observed: `http://short.test` gives `http://short.test/<code>` (test #1), `http://localhost:18082` gives `http://localhost:18082/<code>`, and `http://localhost:18083/` gives `http://localhost:18083/<code>`.

How the existing code is found for a duplicate. `UrlServiceIMPL.createShortUrl` first validates the input. It then iterates over `urlMappings.entrySet()` (a `ConcurrentHashMap<String code, String url>`) and compares each stored value with the raw input using `String.equals`. The input is not normalised (no trimming, case folding or trailing-slash handling), so `.../42/?ref=manual` and `.../42?ref=manual` get different codes (curl X3). The key of the first matching entry is returned as the existing code.

How the controller decides between 201 and 200. The service returns `CreateUrlResult(CreateUrlResponse response, boolean created)`. `created` is `false` when the scan found an existing entry and `true` when a new code was generated and stored. The controller returns `201 CREATED` when `response.created()` is true and `200 OK` otherwise, with the same `CreateUrlResponse` body in both cases.

How each failure maps to an HTTP status.

Failure	Raised where	Handled by	Status / body
`url` missing (`{}`), `null`, empty or whitespace-only	`UrlValidator` line 10, `InvalidUrlException("URL cannot be empty")`	`GlobalExceptionHandler.handleInvalidUrlException`	400 `{"message":"URL cannot be empty"}`
`URI.create` throws (illegal characters such as a space, or `https://` with no authority)	`UrlValidator` line 22, `InvalidUrlException("Malformed URL")`	`GlobalExceptionHandler.handleInvalidUrlException`	400 `{"message":"Malformed URL"}`
No scheme, a scheme other than http/https, or no host	`UrlValidator` line 19, `InvalidUrlException("Invalid URL format")`	`GlobalExceptionHandler.handleInvalidUrlException`	400 `{"message":"Invalid URL format"}`
Empty request body or malformed JSON	Spring `HttpMessageNotReadableException`	Spring `DefaultHandlerExceptionResolver` (one WARN line)	400 Spring default body `{"timestamp","status","error","path"}`

Failure	Raised where	Handled by	Status / body
Unknown code (`GET /{code}` with a code not in the map)	`UrlServiceIMPL` line 55, `InvalidCodeException("Unknown code")`	`GlobalExceptionHandler.handleInvalidCodeException`	404 `{"message":"Unknown code"}`, nothing logged
`GET /` (no code)	No handler matches	Spring	404 Spring default body, no stack trace (section 7.5)

The exact URL validation rules (`UrlValidator.validateUrl`, applied in order to the raw string):

1. Reject with "URL cannot be empty" if the value is `null` or `isBlank()` (empty, or whitespace only).
2. Parse with `java.net.URI.create(value)`. If it throws `IllegalArgumentException` (any RFC 2396 syntax error as implemented by `java.net.URI`, including unencoded spaces, leading or trailing whitespace, and `https://` with no authority), reject with "Malformed URL".
3. Reject with "Invalid URL format" if `getScheme()` is `null`, or is neither `http` nor `https` (compared case-insensitively).
4. Reject with "Invalid URL format" if `getHost()` is `null`. `java.net.URI` returns `null` for a missing host and for an authority it cannot parse as a server-based host.
5. Nothing else is checked: there are no length, port-range, userinfo, TLD or non-ASCII rules. The original string, not a normalised form, is what gets stored.

Authorship

- **This session (revisions 1 to 3):** I did **not** write, edit, reformat or delete any file under `src/main`, `pom.xml`, `application.yaml`, `.gitignore` or any other config. The author made every `src/main` and `pom.xml` change between revisions: the 404 fix, `buildShortUrl`, removal of the dead check and the unused imports, and removal of the compiler-plugin block.
- **Files I created:** `UrlShortenerErrorCasesTest.java` (revision 1) and `ShortUrlBaseUrlTest.java` (after revision 2, at the author's request), both under `src/test`, plus the two report files in `docs/reviews/`. Build outputs went to `target/`, and scratch scripts and logs went to a temporary directory outside the repo.
- **Git:** I did not stage, commit or push anything. The author made commits `6fea8bb`, `1607d0a` and `665908a`.
- **Earlier sessions:** I can't see earlier sessions, so I can't say whether Claude wrote or edited any `src/main` code in them. The repository's `CLAUDE.md` (added in `cd61f9d`) tells Claude never to create or edit files under `src/main`. All commits are authored by `malinda-sampath`.